

Machine Learning Through the Lens of Representation

Mohammad Mahdi Nasimi

Contents

1	The Broken Toy	2
1.1	The Phenomenon	2
1.2	The Broken Toy: A Cantilever Beam	3
1.3	The Failure	4
1.4	The Representational Shift	5
1.5	Geometry and Intuition	5
1.6	From the Physical World to Raw Data	6
1.7	The Representation Pipeline	7
1.8	Translation to Code	8
1.9	What the Cantilever Taught Us	9
1.10	The Same Lesson, Twice	9
1.11	Connection to Next Session	10
1.12	Exercises (Ungraded)	10

1 The Broken Toy

Session 1

One-line motivation. Raw coordinates are a projection. They mix causes, hide structure, and mislead algorithms. Before we can learn anything useful, we must ask what representation the phenomenon actually deserves.

Session 0 gave us the projectile. The failure there was that our hypothesis class could not express the shape of the phenomenon. The cantilever is a different failure with the same root cause: the *coordinates themselves* mix the causes. In both cases, the fix was to change the representation, and in both cases, no amount of data or tuning could have saved the original setup.

1.1 The Phenomenon

We begin with a question that has nothing to do with machine learning, and everything to do with it.

What are these two tables in Figure 1?

To a computer, they are nothing but two grids of real numbers. Each entry is a pixel intensity, a raw sensor reading, a single coordinate in a high-dimensional space.

64	68	55	70	76	64	67	74	70	70	76	81	84	89	82	74
60	67	148	158	151	95	73	77	71	72	76	74	74	138	77	69
65	104	173	148	174	133	75	77	71	82	91	92	101	136	132	130
66	73	124	169	135	146	85	89	96	117	131	143	149	171	162	161
81	77	80	114	144	181	208	194	194	201	200	197	181	192	158	156
112	113	120	105	112	176	202	202	206	192	154	157	196	176	138	136
140	142	140	145	133	168	191	183	171	153	129	152	180	177	136	138
139	147	136	164	152	163	176	154	135	115	68	111	184	168	115	114
137	134	127	137	85	117	145	114	102	117	75	77	180	170	124	117
106	112	113	112	111	93	166	135	125	124	111	85	144	164	115	117
114	113	113	115	133	111	158	135	127	129	126	97	115	129	118	118
130	132	114	121	135	122	152	136	132	123	122	125	112	134	122	120
114	116	122	131	137	125	148	135	130	121	116	111	117	125	124	109
109	108	115	137	135	113	164	113	110	105	105	100	117	134	131	108
116	115	112	118	96	102	143	104	110	108	103	104	102	103	109	114
116	118	114	115	114	114	111	116	106	112	108	120	124	121	119	115

179	128	83	95	75	55	61	106	121	97	51	55	54	52	40	76
121	109	77	62	40	54	78	139	175	171	43	48	51	48	38	54
172	160	136	145	159	162	155	147	136	151	126	122	120	122	117	112
149	152	151	158	157	157	155	136	108	141	157	153	150	150	147	142
143	146	142	142	154	103	64	113	105	132	101	139	138	137	138	137
141	141	141	143	133	41	55	101	142	140	62	88	136	136	133	130
142	134	142	144	112	28	51	77	114	123	53	71	126	123	121	127
130	126	127	136	85	17	32	43	118	76	33	70	135	122	117	118
126	118	122	139	66	18	28	27	106	31	18	63	135	132	125	113
125	117	115	122	39	19	28	22	39	23	39	64	176	187	114	119
130	116	114	111	35	26	28	21	21	26	68	62	186	176	108	104
119	114	107	103	32	27	27	26	23	37	76	64	186	163	145	105
117	106	97	99	123	35	40	45	29	55	97	90	155	136	110	104
111	93	93	93	84	33	33	45	31	69	107	116	135	125	101	101
99	95	95	82	79	36	30	34	48	79	96	103	128	118	93	89
97	89	88	88	82	37	61	147	96	88	82	66	139	150	94	88

Figure 1: Two raw images as arrays of pixel intensities. On the left, a dog. On the right, a man. As numbers, they are structurally indistinguishable.

At first glance the two arrays look indistinguishable. They are the same size, they have the same range of values, and if you shuffled their entries you could not tell which one came from which. Yet when we look at them, we do not see numbers. We see a dog and a man. Nothing in the raw numbers themselves announces “dog” or “man”. The meaning lives entirely in the *pattern*—in how the numbers are arranged relative to one another.

What machine learning does, at its core, is learn to read such patterns. It takes a grid of numbers and produces a label, a category, a decision. And remarkably, it can do this even when the

underlying structure is invisible to us in the raw form. The gap between the numbers and the meaning is exactly the gap that learning algorithms are designed to bridge.



Figure 2: The same two arrays, now rendered in color. Structure that was invisible in the raw numeric form—edges, contours, regions—becomes visible. The numbers have not changed. Only the representation has.

When we render the same arrays in color—mapping each numeric value to a hue—the underlying structure becomes visible. Regions that share similar values light up with similar colors; edges and contours emerge from the noise. The numbers have not changed. Only our representation of them has. This is a small but important lesson: *the same data can look meaningless or meaningful depending entirely on how we choose to present it*, and much of the art of machine learning lies in choosing representations that make the hidden structure visible.

*The raw representation is semantically empty. The same numbers could be a dog, a man, a galaxy, or noise. **Richness of representation is what makes meaning possible.***

This is not a trick. It is the central problem of the entire course. A dog and a man are distinguishable in the world. They are not distinguishable in their raw pixel coordinates. The gap between the phenomenon and the data is where machine learning lives.

But recognizing this gap is not enough. The real question—the one that will occupy us for the rest of the course—is this:

***How can we manipulate data with respect to its structure?** How do we transform a raw grid of numbers into a representation in which the structure we care about is not merely present, but visible, usable, and connected to a decision?*

1.2 The Broken Toy: A Cantilever Beam

Now we make the question concrete. Consider a cantilever beam with two strain gauges at the same cross-section:

- one on the **top surface**, reading ε_{top} ,
- one on the **bottom surface**, reading $\varepsilon_{\text{bottom}}$.

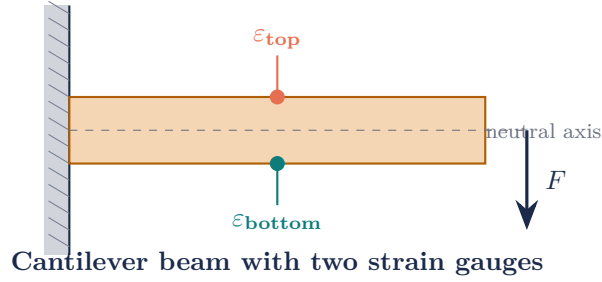


Figure 3: A cantilever beam with two strain gauges. The neutral axis experiences no bending strain.

The beam can experience two independent physical effects:

- **Axial strain** ε_a : tension or compression. Same sign on top and bottom.
- **Bending strain** ε_b : bending. Opposite signs on top and bottom.

The raw sensor vector is

$$\mathbf{x}_{\text{raw}} = \begin{bmatrix} \varepsilon_{\text{top}} \\ \varepsilon_{\text{bottom}} \end{bmatrix}.$$

But the physically meaningful coordinates are

$$\varepsilon_a = \frac{\varepsilon_{\text{top}} + \varepsilon_{\text{bottom}}}{2}, \quad \varepsilon_b = \frac{\varepsilon_{\text{top}} - \varepsilon_{\text{bottom}}}{2}.$$

These coordinates come from the mechanics of beam deformation, not from the data. They are supplied by physics, in the same way that the sinusoidal basis in Session 0 was supplied by the physics of projectile motion. Finding the right representation is not a purely data-driven task; it requires prior knowledge about the phenomenon.

In matrix form:

$$\begin{bmatrix} \varepsilon_{\text{top}} \\ \varepsilon_{\text{bottom}} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}}_{\mathbf{A}} \begin{bmatrix} \varepsilon_a \\ \varepsilon_b \end{bmatrix} + \boldsymbol{\eta}, \quad (1)$$

where $\boldsymbol{\eta}$ is measurement noise.

Remark 1.1: What Equation (1) Says

The raw measurement is a *rotated and scaled* view of the true physical state. The matrix $\mathbf{A}$ is invertible, so no information is lost in principle. But the coordinates are *mixed*: each raw reading depends on both physical causes.

1.3 The Failure

Suppose our task is to detect bending. We collect N samples, label each by whether $\varepsilon_b > 0$, and train a shallow decision tree on the raw coordinates $(\varepsilon_{\text{top}}, \varepsilon_{\text{bottom}})$.

The tree partitions the plane with axis-aligned splits. But the physical boundary $\varepsilon_b = 0$ corresponds to

$$\varepsilon_{\text{top}} - \varepsilon_{\text{bottom}} = 0 \iff \varepsilon_{\text{top}} = \varepsilon_{\text{bottom}},$$

which is a *diagonal* line in the raw plane.

A decision tree cannot represent a diagonal boundary without a staircase of many axis-aligned splits. The model is not stupid. **The coordinates are stupid.**

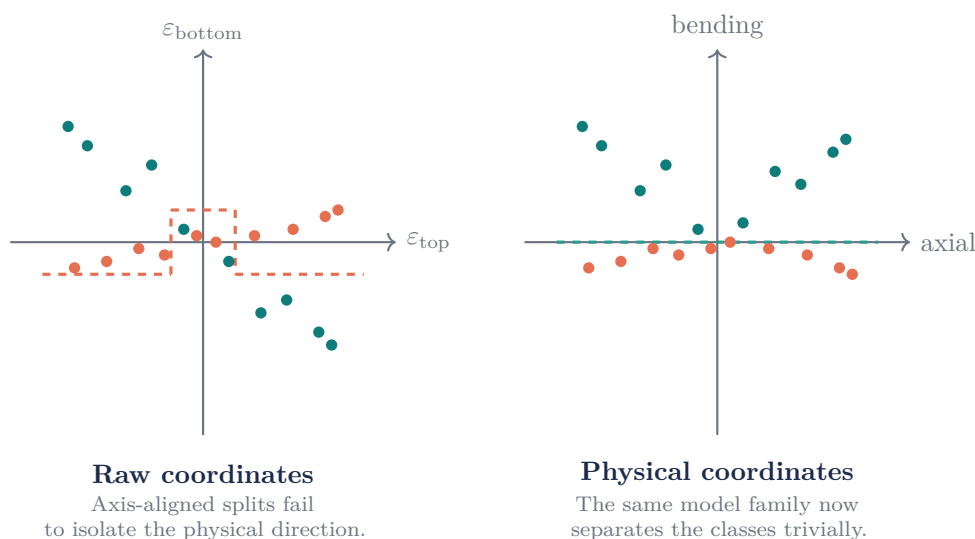


Figure 4: Left: raw coordinates force axis-aligned approximations. Right: the same data in physical coordinates is trivially separable.

1.4 The Representational Shift

Now rotate the coordinates:

$$\text{axial} = \frac{\varepsilon_{\text{top}} + \varepsilon_{\text{bottom}}}{2}, \quad \text{bending} = \frac{\varepsilon_{\text{top}} - \varepsilon_{\text{bottom}}}{2}.$$

In these coordinates, the bending problem becomes a one-dimensional threshold. The same model family, with the same capacity, now works with almost no effort.

The model did not get smarter. The representation got honest. Raw coordinates are not the problem—they are a coordinate system, and it was the wrong one.

1.5 Geometry and Intuition

What just happened, geometrically?

- The physical state lives in a 2D space spanned by $(\varepsilon_a, \varepsilon_b)$.
- The observation map is a linear transformation $\mathbf{A}$ plus noise.
- The task depends only on ε_b , i.e., on one coordinate in the physical basis.

- In the raw basis, this coordinate is a linear combination of both measurements.

The failure was not a failure of the model class. It was a failure to align the coordinate system with the structure of the phenomenon.

Definition 1.1: Change of Basis

Let $\mathbf{x} \in \mathbb{R}^n$ be a raw observation and let $\mathbf{W} \in \mathbb{R}^{n \times n}$ be invertible. The representation $\mathbf{z} = \mathbf{W}\mathbf{x}$ is a change of basis. The structure of the problem is preserved, but its *visibility* to a given model class may change dramatically.

1.6 From the Physical World to Raw Data

Before any learning can happen, we must collect data from the physical world. Let $\mathcal{W}$ denote the continuous physical reality. Computers cannot process infinite-dimensional systems directly, so sensors, cameras, or measurement devices discretize $\mathcal{W}$ into a finite dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N \subset \mathcal{X} \times \mathcal{Y}$, where $x_i \in \mathbb{R}^D$ is a feature vector of raw measurements and $y_i \in \mathcal{Y}$ is the corresponding target.

Observation is never exact. Sensor precision, unobserved variables, and environmental noise all corrupt the measurement. We never see the true relationship $f(x)$ directly; we only see noisy samples.

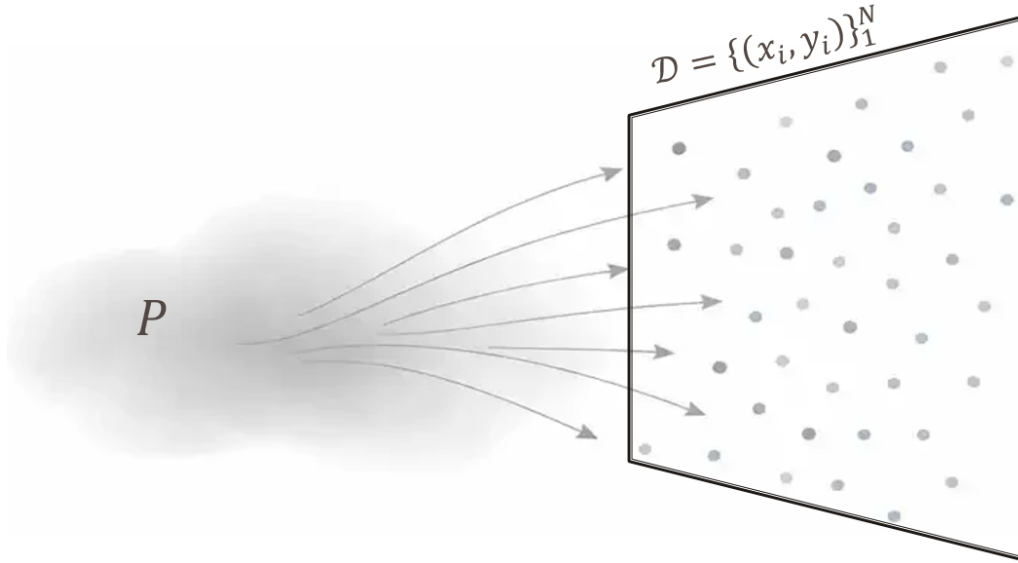


Figure 5: Continuous physical world $\mathcal{W}$ projected onto a discrete observation plane $\mathcal{D}$.

A digital photograph is the simplest example: the continuous scene is discretized into a grid of pixels, each with finite intensity. Learning happens on these discrete points, not on continuous reality.

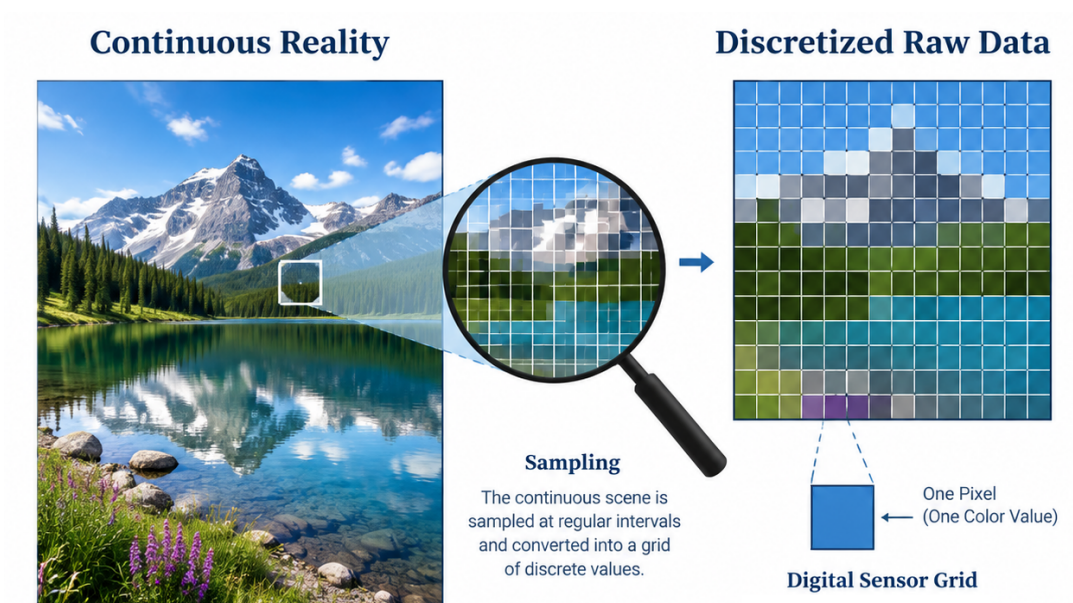


Figure 6: Continuous reality to discretized data.

1.7 The Representation Pipeline

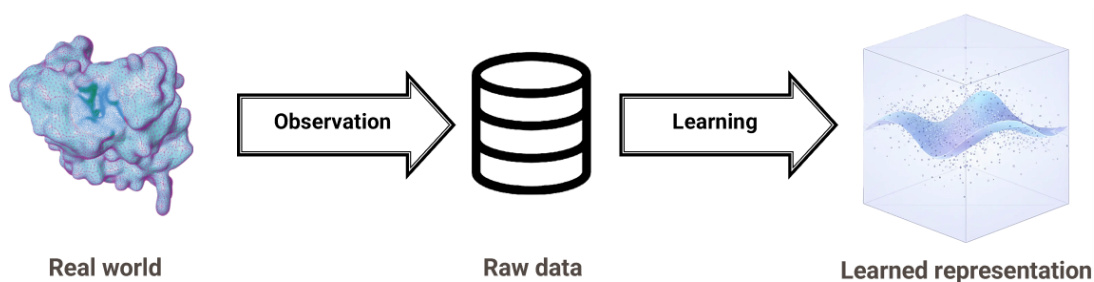
Every problem in this course follows the same ontology. A learning system is a composition of two maps:

1. **Physical Observation Mapping $\mathcal{O}$:** the environment $\mathcal{W}$ is sampled to form the raw dataset $\mathcal{D}$.

$$\mathcal{O} : \mathcal{W} \longrightarrow \mathcal{D} \quad (2)$$

2. **Representation Mapping $\mathcal{A}$:** a learning algorithm maps raw features into a learned secondary representation space $\mathcal{Z}$.

$$\mathcal{A} : \mathcal{D} \longrightarrow \mathcal{Z} \quad (3)$$

Figure 7: A learning system is a composition of two maps ($\mathcal{W} \xrightarrow{\mathcal{O}} \mathcal{D} \xrightarrow{\mathcal{A}} \mathcal{Z}$).

The phenomenon has structure, symmetry, and useful information *before* we ever observe it. Observation is a lossy projection. The algorithm is a second projection. The secondary representation is meaningful only if it preserves what the first projection did not destroy.

Definition 1.2: The Representation Problem

Given a phenomenon P with structure $\mathcal{S}$, an observation map $o : P \rightarrow \mathbb{R}^n$, and a task T , find a representation $\phi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ such that the structure relevant to T is preserved or made explicit in the coordinates $\phi(o(P))$.

The rest of this course is the systematic study of ϕ .

1.8 Translation to Code

Listing 1: Generating and visualizing the cantilever data.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 rng = np.random.default_rng(0)
5 n = 300
6
7 axial    = rng.normal(0, 1.0, n)
8 bending  = rng.normal(0, 1.0, n)
9 noise    = 0.05 * rng.normal(0, 1, (n, 2))
10
11 top      = axial + bending + noise[:, 0]
12 bottom   = axial - bending + noise[:, 1]
13
14 # Raw coordinates
15 plt.figure(figsize=(6, 5))
16 plt.scatter(top, bottom, c=(bending > 0), cmap="coolwarm", s=18)
17 plt.xlabel(r"$\varepsilon_{\mathrm{top}}$")
18 plt.ylabel(r"$\varepsilon_{\mathrm{bottom}}$")
19 plt.title("Raw coordinates")
20 plt.axline((0, 0), slope=1, color="k", linestyle="--", lw=1)
21 plt.show()
22
23 # Physical coordinates
24 axial_c   = (top + bottom) / 2
25 bending_c = (top - bottom) / 2
26
27 plt.figure(figsize=(6, 5))
28 plt.scatter(axial_c, bending_c, c=(bending_c > 0), cmap="coolwarm", s=18)
29 plt.xlabel("axial")
30 plt.ylabel("bending")
31 plt.title("Physical coordinates")
32 plt.axhline(0, color="k", linestyle="--", lw=1)
33 plt.show()

```

Listing 2: The same model, two representations.

```

1 from sklearn.tree import DecisionTreeClassifier
2 from sklearn.model_selection import cross_val_score
3
4 X_raw = np.c_[top, bottom]
5 X_phys = np.c_[axial_c, bending_c]
6 y = (bending > 0).astype(int)
7

```

```

8 clf = DecisionTreeClassifier(max_depth=3, random_state=0)
9
10 score_raw = cross_val_score(clf, X_raw, y, cv=5).mean()
11 score_phys = cross_val_score(clf, X_phys, y, cv=5).mean()
12
13 print(f"Raw coordinates:      {score_raw:.3f}")
14 print(f"Physical coordinates: {score_phys:.3f}")

```

Typical output:

Raw coordinates: 0.72

Physical coordinates: 0.99

Same model. Same capacity. Different coordinates.

1.9 What the Cantilever Taught Us

Let us be careful about what we have actually shown. The cantilever does not prove that all representation problems are solved by rotations. It proves something narrower and more useful: *when the coordinates mix independent causes, and the model has a bias toward axis-aligned structure, the failure is representational, not algorithmic. No amount of data or tuning will fix it.*

This gives us a specific diagnostic for the cantilever's class of problem:

1. What structure does the phenomenon have?
2. What does the observation preserve, and what does it mix?
3. Does the model's inductive bias align with that structure?

If the answer to (3) is no, the model will fail. This is the shape of the failure the cantilever has shown us. In Session 2 we will generalize this into a diagnostic habit that applies to a much wider range of problems.

1.10 The Same Lesson, Twice

Sessions 0 and 1 have now shown the same root cause through two different mechanisms. The projectile's hypothesis class could not express the shape of the phenomenon. The cantilever's coordinates mixed the causes. Both are representation failures. Both were fixed by changing the representation. Both would have been immune to any amount of data or tuning.

	Projectile (Session 0)	Cantilever (Session 1)
Wrong representation	raw (θ, R)	raw $(\varepsilon_{\text{top}}, \varepsilon_{\text{bottom}})$
Failure mode	hypothesis class too simple	coordinates mixed independent causes
Fix	change the features	change the basis
Where the fix came from	Newtonian mechanics	mechanics of beam deformation

Table 1: Two problems, two failures, one cause.

The table is worth staring at. The right column of each row is different. The left column is

identical. In both cases, the answer came from physics, not from the data.

1.11 Connection to Next Session

We have seen that raw coordinates can be misleading, and that a change of basis can rescue a model. But notice something uncomfortable about both examples: in Session 0, physics supplied the shape. In Session 1, physics supplied the coordinates. Both times, we knew the answer before we looked at the data.

What happens when we do not? What happens when we have measurements from a system whose physics we do not understand, and we cannot write down the correct coordinates in advance? Do we have any hope of learning the representation from the data itself?

This is the question of Session 2: **The Hidden Shape**.

1.12 Exercises (Ungraded)

1. **The projectile, revisited.** Take the projectile problem from Session 0. Assume you have data only in $[10^\circ, 30^\circ]$. Fit linear regression, poly-3, and the sin/cos model. Show the extrapolation behavior out to 80° . Explain why the sin/cos model succeeds and the others fail. What role did physics play in each model's success or failure?
2. **The inverse of the observation map.** Derive the inverse of $\mathbf{A}$ in Equation (1). Recover $(\varepsilon_a, \varepsilon_b)$ exactly from $(\varepsilon_{\text{top}}, \varepsilon_{\text{bottom}})$ in the noiseless case.
3. **Noise is not the problem.** Generate the cantilever data with $\eta = 0$. Confirm that a decision tree can still fail on raw coordinates. Why?
4. **A different model class.** Replace the decision tree with a logistic regression classifier. Does the gap between raw and physical coordinates shrink, stay the same, or grow? Explain geometrically.
5. **Discovering the coordinates from data.** The correct coordinates $(\varepsilon_a, \varepsilon_b)$ were supplied by physics. Suppose they were not. Can we recover them from the data alone?

Take the raw cantilever data. Standardize it and run PCA:

```

1 from sklearn.decomposition import PCA
2 from sklearn.preprocessing import StandardScaler
3
4 X = np.c_[top, bottom]
5 X_std = StandardScaler().fit_transform(X)
6
7 pca = PCA(n_components=2).fit(X_std)
8 print("Principal directions:")
9 print(pca.components_)
10 print("Explained variance ratio:",
11       pca.explained_variance_ratio_)
12
13 Z = pca.transform(X_std)
14 plt.scatter(Z[:, 0], Z[:, 1], c=(bending > 0),
15           cmap="coolwarm", s=18)
16 plt.xlabel("PC 1")
17 plt.ylabel("PC 2")
18 plt.show()
```

Look at the principal directions. Do they correspond to the axial and bending axes? Why or why not? What does this tell you about the relationship between PCA and prior knowledge?

This is a preview of Session 2.

6. **Transfer task.** Consider two thermocouples on opposite sides of a wall:

$$T_{\text{left}} = T_{\text{mean}} + T_{\text{gradient}}, \quad T_{\text{right}} = T_{\text{mean}} - T_{\text{gradient}}.$$

Repeat the entire analysis. What is the physical meaning of the rotation?

7. **Reflection.** In your own engineering discipline, identify one measurement that mixes two physically distinct causes. Write one paragraph describing the raw representation and the physical representation.